

C.4 问题四：峰值资源核算与完整分区枚举

代码清单10先以半开区间事件扫描计算每类资源的峰值并发数，再从运输共访关系形成原子块。分区时固定第一个原子块的标签，遍历其余标签并排除空组，按总缺口、总配置、工作量变异系数和完成时刻极差选择方案，最后逐类型输出缺口与相对集中执行的冗余。

Listing 10 问题四资源峰值与两组、三组分区枚举

```
2092 def _max_overlap(intervals: list[tuple[float, float]]) -> int:
2093     events = [(start, 1) for start, end in intervals if end > start] + [
2094         (end, -1) for start, end in intervals if end > start
2095     ]
2096     active = best = 0
2097     for _, delta in sorted(events, key=lambda x: (x[0], x[1])):
2098         active += delta
2099         best = max(best, active)
2100     return best
2101
2102
2103 def _solve_q4_partitions(
2104     data: dict[str, Any],
2105     q3_trips: list[dict[str, Any]],
2106     relay_missions: list[dict[str, Any]],
2107     communication_rows: list[dict[str, Any]],
2108 ) -> tuple[list[dict[str, Any]], list[dict[str, Any]]]:
2109     services = sorted(k for k in data["nodes"] if k.startswith("S"))
2110     parent = {s: s for s in services}
2111
2112     def find(x: str) -> str:
2113         while parent[x] != x:
2114             parent[x] = parent[parent[x]]
2115             x = parent[x]
2116         return x
2117
2118     def union(a: str, b: str) -> None:
2119         ra, rb = find(a), find(b)
2120         if ra != rb:
2121             parent[max(ra, rb)] = min(ra, rb)
2122
2123     for trip in q3_trips:
2124         for service in trip["route"][1:]:
2125             union(trip["route"][0], service)
2126     components: dict[str, list[str]] = {}
2127     for service in services:
2128         components.setdefault(find(service), []).append(service)
2129     blocks = sorted(components.values(), key=lambda x: x[0])
2130
2131     inventory = {
2132         "A_uav": 4, "B_uav": 2, "C_uav": 2,
2133         "A_battery": 6, "B_battery": 4, "C_battery": 4,
2134         "relay_uav": data["relay_inventory"]["units"],
2135         "relay_pack": data["relay_inventory"]["packs"],
2136     }
2137
2138     # A relay mission belongs to a partition only when the final communication
2139     # segmentation actually assigns that mission to one of the partition's
2140     # transport trips. ``served_trip_ids`` is the wave-level candidate list
2141     # used during Q3 coverage search and can include trips that remained on a
2142     # direct link or were assigned to another relay; using it here would copy
2143     # unnecessary relay tasks into Q4 and overstate independent resources.
2144     actual_trips_by_mission: dict[str, set[str]] = {}
2145     for row in communication_rows:
2146         mission_id = row.get("relay_mission_id")
2147         if mission_id:
2148             actual_trips_by_mission.setdefault(str(mission_id), set()).add(str(row["trip_id"]))
2149
2150     def resources_for(group: set[str]) -> tuple[dict[str, int], float, float]:
2151         trips = [x for x in q3_trips if set(x["route"]) & group]
2152         trip_ids = {x["trip_id"] for x in trips}
2153         missions = [
2154             x for x in relay_missions
2155             if trip_ids & actual_trips_by_mission.get(x["mission_id"], set())
2156         ]
```